

Timers, Timing and PWM

Introduction to ARM Cortex-M Microprocessors – Chapter 10

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

- 1 **Why This Matters**

- 2 **Why Software Delays Fail**

- 3 **Timer Architecture**

- 4 **Alternate Functions**

- 5 **PWM**

`delay_ms()` has three problems. Its timing changes with the optimizer and the clock. It uses the CPU while it waits. It gives no accurate time base. A hardware timer solves all three. It counts on its own, so it can **schedule** an event, **measure** an event, or **generate** a timed output such as PWM.

After this chapter, you can:

1. Explain how prescaler and reload set a timer's period.
2. Calculate prescaler and reload values.
3. Generate periodic update interrupts.
4. Keep a millisecond time base with SysTick.
5. Choose the alternate function that routes a pin to a timer channel.
6. Calculate PWM frequency and duty cycle.
7. Apply PWM to LED brightness and motor speed.

- 1 Why This Matters
- 2 **Why Software Delays Fail**
- 3 Timer Architecture
- 4 Alternate Functions
- 5 PWM

Three Things Wrong With `delay_ms()`

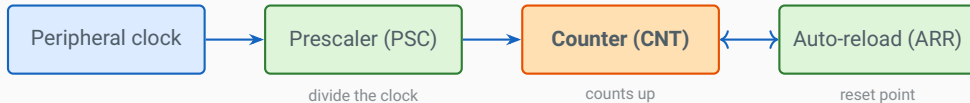
```
void delay_ms(int n) {  
    for (volatile int i = 0;  
        i < n * 4000; i++) ;  
}
```

1. Its timing changes with the **compiler optimisation level** and the **clock frequency**
2. It consumes **100%** of the CPU while waiting
3. It cannot be **interrupted accurately** – any ISR that fires makes the delay longer

A timer is a counter in hardware

It runs independently of the processor, at a rate you configure. It does not care what the CPU is doing, what the optimiser did, or how many interrupts fired. **That is what makes it a clock rather than a guess.**

- 1 Why This Matters
- 2 Why Software Delays Fail
- 3 **Timer Architecture**
- 4 Alternate Functions
- 5 PWM



$$f_{\text{event}} = \frac{f_{\text{CK}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$

The **+1** is because a value of 0 means “divide by 1” – a counter that reloads at 0 still counts one tick.

Required

A 1 Hz interrupt from a 16 MHz clock.

Choose a prescaler

PSC = 15999 divides 16 MHz down to **1 kHz** — a convenient 1 ms tick.

Then the reload value

ARR = 999 gives 1000 ticks = **1 s**.

$$\text{Check: } \frac{16,000,000}{16000 \times 1000} = 1 \text{ Hz.}$$

TIM2->PSC = 15999;

TIM2->ARR = 999;

Why choose PSC first

Both registers are 16-bit, so neither can exceed 65535. Pick the prescaler to give a **convenient tick** — 1 ms or 1 μ s — then the reload value is just the number of ticks you want.

```
RCC->APB1ENR |= (1U << 0);    // TIM2 clock
TIM2->PSC = 15999;             // 16 MHz -> 1 kHz
TIM2->ARR = 999;               // 1000 ticks -> 1 Hz
TIM2->DIER |= (1U << 0);       // update interrupt enable
TIM2->CR1 |= (1U << 0);        // start counting
NVIC_EnableIRQ(TIM2_IRQn);

void TIM2_IRQHandler(void)
{
    if (TIM2->SR & (1U << 0)) { // update flag?
        TIM2->SR &= ~(1U << 0); // clear it
        GPIOA->ODR ^= (1U << 5);
    }
}
```

The CPU is now **free** between interrupts — it can sleep, or do real work, and the LED still blinks at exactly 1 Hz.

SysTick The system time base

- A **24-bit** down-counter built into the **core**, not the vendor's peripherals.
- Present on **every** Cortex-M, at the same address — so code using it is portable.
- Normally configured for a **1ms** tick.

```
SysTick_Config(SystemCoreClock / 1000);
```

```
volatile uint32_t ticks = 0;

void SysTick_Handler(void)
{
    ticks++;
}

uint32_t millis(void)
{
    return ticks;
}
```

This is what replaces `delay_ms()`

Instead of *waiting*, record when something last happened and check whether enough time has passed. That is the **non-blocking** pattern Chapter 14 builds a whole system from.

```
void delay_ms_bad(int n)
{
    for (volatile int i = 0; i < n * 4000; i++)
        ;
    /* Wrong in three ways: drifts with optimisation level
       and clock frequency, burns 100% CPU, and any ISR that
       fires makes it longer than requested. Do not use this
       in real designs. */
}
```

A Proper `delay_ms()` — Built on SysTick

```
static volatile uint32_t systick_ms = 0;

void SysTick_Handler(void) { systick_ms++; }

void systick_init(uint32_t core_clock_hz)
{
    SysTick_Config(core_clock_hz / 1000U); // 1 ms tick
}

uint32_t millis(void) { return systick_ms; }

// Blocking, but exact regardless of compiler optimisation:
// it counts real elapsed ticks, not a fixed loop count.
void delay_ms(uint32_t n)
{
    uint32_t start = millis();
    while ((millis() - start) < n) {
        __asm volatile ("wfi"); // sleep between ticks
    }
}
```

An Alternative: `delay_ms()` on a General-Purpose Timer

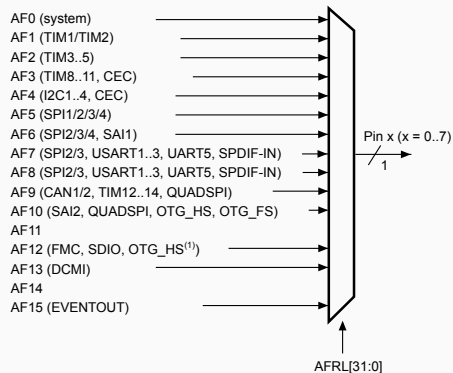
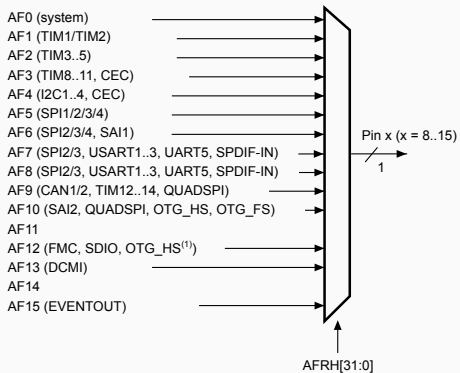
```
void tim2_delay_init(void)
{
    RCC->APB1ENR |= (1U << 0); // TIM2 clock enable
    TIM2->PSC = 15999;          // 16 MHz -> 1 kHz
    TIM2->ARR = 0xFFFF;         // free-running
    TIM2->CR1 |= (1U << 0);      // start counting
}

void delay_ms_tim2(uint32_t n)
{
    TIM2->CNT = 0;
    while (TIM2->CNT < n)
        ;
}
```

Useful when SysTick is already claimed by an RTOS, and the mechanism is the same prescaler/auto-reload arithmetic from earlier in this chapter.

- 1 Why This Matters
- 2 Why Software Delays Fail
- 3 Timer Architecture
- 4 Alternate Functions**
- 5 PWM

Routing a Pin to a Peripheral



A pin can serve several peripherals. MODER set to 10 selects alternate-function mode; AFR then chooses which of up to sixteen functions.

Selecting the Function Number

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Four bits per pin, so `AFR[0]` covers pins 0–7 and `AFR[1]` covers pins 8–15.

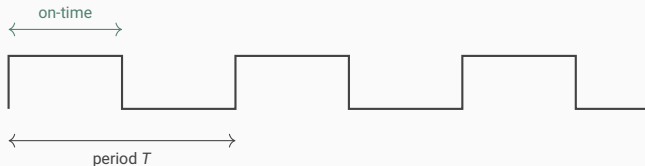
For pin $n \geq 8$ the shift is $(n - 8) \times 4$.

```
// PA5 -> TIM2_CH1 is AF1
GPIOA->MODER &= ~(3U << 10);
GPIOA->MODER |= (2U << 10);

GPIOA->AFR[0] |= (1U << (5*4));
```

The function numbers are in the **datasheet**, not the reference manual – a distinction that costs people an afternoon.

- 1 Why This Matters
- 2 Why Software Delays Fail
- 3 Timer Architecture
- 4 Alternate Functions
- 5 **PWM**



$$\text{duty cycle} = \frac{\text{on-time}}{T} \quad f_{\text{PWM}} = \frac{f_{\text{CK}}}{(\text{PSC} + 1)(\text{ARR} + 1)}$$

A digital pin producing an analog effect

The pin is only ever fully on or fully off — but switch it fast enough and an LED looks **dimmed**, a motor runs **slower**, a heater delivers **less power**. The average is what the load responds to.

```
TIM2->PSC  = 15;           // 16 MHz -> 1 MHz
TIM2->ARR   = 999;          // 1 MHz / 1000 = 1 kHz PWM
TIM2->CCR1  = 250;          // 250/1000 = 25% duty

TIM2->CCMR1 |= (6U << 4);   // PWM mode 1 on channel 1
TIM2->CCMR1 |= (1U << 3);   // preload enable
TIM2->CCER  |= (1U << 0);   // enable the output
TIM2->CR1   |= (1U << 0);   // start
```

ARR sets the frequency, CCR sets the duty

Change CCR1 at any time and the duty cycle changes on the next cycle — no glitch, because the preload register updates only at the reload point. **That is what makes smooth dimming possible.**

A higher PWM frequency is less visible but wastes more switching energy. For LEDs, 1 kHz is comfortably above flicker perception.

```
void pwm_init_tim2_ch1(uint32_t psc, uint32_t arr)
{
    RCC->APB1ENR |= (1U << 0);
    TIM2->PSC = psc;
    TIM2->ARR = arr;
    TIM2->CCR1 = 0;           // start at 0% duty
    TIM2->CCMR1 |= (6U << 4); // PWM mode 1, channel 1
    TIM2->CCMR1 |= (1U << 3); // preload enable
    TIM2->CCER |= (1U << 0);  // enable output
    TIM2->CR1  |= (1U << 0);  // start
}

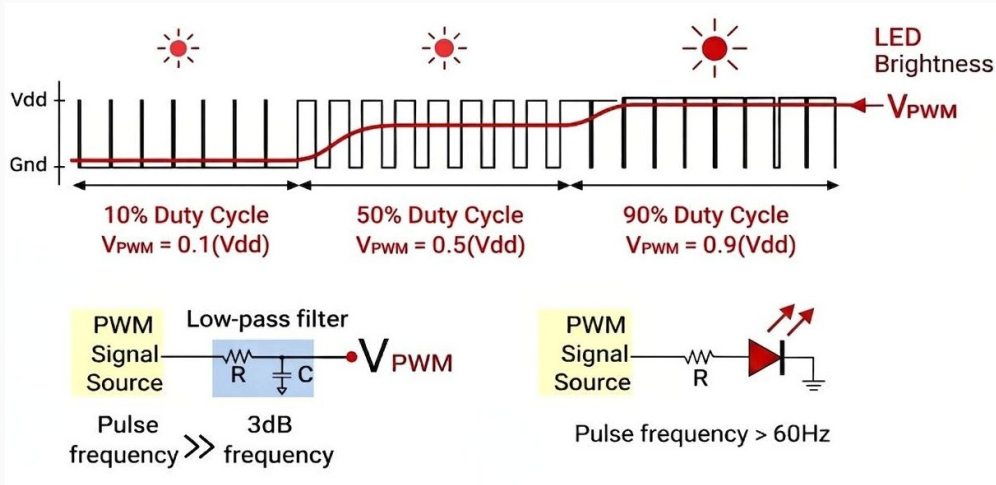
// percent: 0-100
void pwm_set_duty_percent(uint8_t percent)
{
    if (percent > 100) percent = 100;
    uint32_t arr = TIM2->ARR;
    TIM2->CCR1 = ((uint32_t)percent * (arr + 1)) / 100U;
}
```

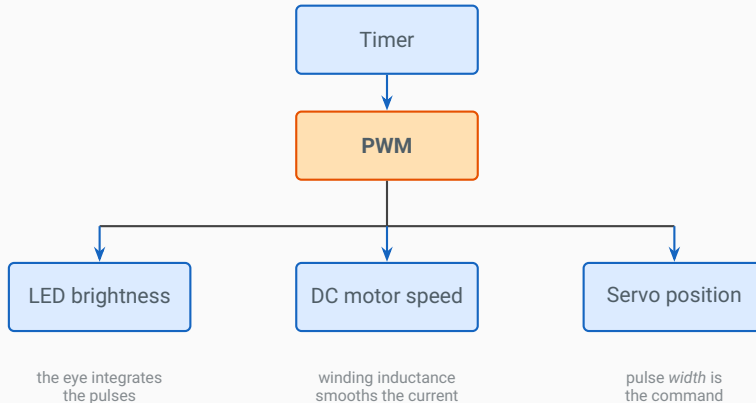
Example: A Breathing LED, Non-Blocking

```
void led_breathe_demo(void)
{
    static periodic_task_t step = { .last_ms = 0, .period_ms = 20 };
    static int duty = 0;
    static int dir = 1;

    if (task_due(&step)) {
        duty += dir;
        if (duty >= 100 || duty <= 0) dir = -dir;
        pwm_set_duty_percent((uint8_t)duty);
    }
}
```

Every 20 ms the duty cycle steps by one percentage point up or down, driven entirely by `task_due()` — no blocking call anywhere in the loop.



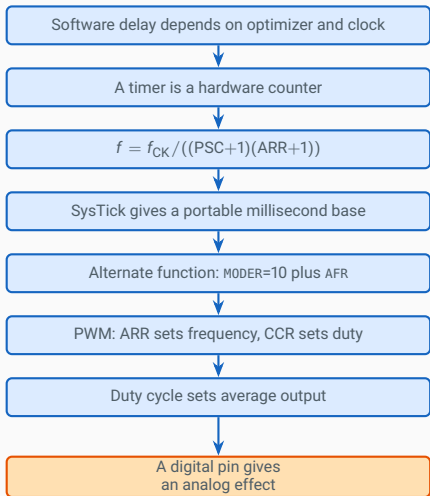


LED and motor rely on **physical averaging**. A servo is different: its control electronics measure the **pulse width** and use it to set a position.

Level	Representation
Physical hardware	Counter, comparator and output flip-flop driving the pad
Register	TIM2_CCR1 compared against TIM2_CNT
C	TIM2->CCR1 = 500;
Assembly	MOV R0, #500 / STR R0, [R1, #0x34]
Observable	A 1 kHz square wave at 50% duty on pin PA5

So far

ARR determines the PWM **period**. CCRx determines how long the output stays **high**. The same two registers used earlier to schedule an interrupt now generate a waveform instead.



Terminology covered

- Prescaler (PSC), reload (ARR)
- SysTick
- Alternate function, AFR
- PWM, duty cycle, CCR

Next: Chapter 11

ADC and Analog Interfacing

We made an analog effect. Next, we measure a real one.